

Formal Technical Process Specification and Verification for Automated Production Systems

Georg Hackenberg¹, Alarico Campetelli¹, Christoph Legat², Jakob Mund¹,
Sabine Teuff³, and Birgit Vogel-Heuser²

¹ Chair IV: Software & Systems Engineering, Technische Universität München,
Boltzmannstr. 3, 85748 Garching, {hackenbe,mund,campetel}@in.tum.de

² Institute of Automation and Information Systems, Technische Universität
München, Boltzmannstr. 5, 85748 Garching, {legat,vogel-heuser}@ais.mw.tum.de

³ fortiss GmbH, An-Institut Technische Universität München, Guerickestr. 25, 80805
München, teufl@fortiss.org

Abstract. The complexity of automated production systems increases constantly due to growing functional requirements and engineering discipline integration. Early design steps include the cross-discipline specification of the system’s technical process, while later steps have to ensure compatibility with the specification. Current specification techniques are able to describe and analyse certain properties on the specification level, however verification of the implementation with respect to the specification is a costly task. To overcome this situation we propose a formal modeling technique, which enables automatic verification of the implementation. We demonstrate the approach on a lab-sized automated production system and finally discuss its advantages and disadvantages.

Keywords: Automated production systems, technical process, formal method.

1 Introduction

Automated production systems are complex mechatronic systems whose engineering comprises various disciplines, e.g. mechanical, electrical and software engineering. In the concurrent engineering process, efficient and effective collaboration between interdisciplinary engineering teams is important towards the projects’ success [19, 20]. Ensuring the correctness of a production system’s design manually, i.e. ensuring the compliance with the technical process, is cumbersome and costly. For this reason, verification is performed only sparsely in practice increasing project risks drastically. Automating the verification would enable more frequent design examination for early detection and correction of design flaws. Therefore, ensuring the correctness of the design by automatic verification is one option to leverage a project’s success.

Various approaches for automatic verification exist, ranging from design-time to runtime techniques. For design-time verification, modeling tools provide specific extensions for formal analyses, e.g. Simulink Design Verifier⁴ or SCADE De-

⁴ <http://www.mathworks.de/products/simulink/>

sign Verifier⁵ which allow to specify desired properties using temporal operators and assertions. In contrast, *Rhapsody in C++* [23] supports verification of UML models, while properties can be formulated using temporal patterns or a graphical notation called Life Sequence Charts [14]. Alternatively, Hugo/RT [4] offers UML communication diagrams for property definition. In contrast to design-time techniques, runtime techniques include runtime verification [6] and online/offline monitoring [22]. Both runtime verification and monitoring allow to express properties using for example temporal logics. While existing approaches already support to examine a system’s design with respect to desired properties both at design-time and at runtime, provided formalisms are not designed to be used within the automation domain (e.g. by process engineers). Rather, dedicated technical process modeling techniques are required providing more suitable modeling vocabulary.

For technical process modeling various modeling notations exist. In particular, the application of notations originally developed in computer science are recently under investigation in the field of automated production systems. For example Zor et al. [27] explore the adaption of the Business Process Model and Notation (BPMN) through domain-specific extensions such as parts flow connectors and material gateways. However, while intended for documentation purposes their approach lacks a formal foundation making automatic verification infeasible. In contrast, Dijkman et al. [15] propose formal semantics for standard BPMN, but lack the necessary domain-specific extensions. Alternatively, the Formalized Process Description [1] provides a standard for technical process specification in the manufacturing domain. While in combination with a plant model the formalism is well suited for diagnosis [13], automatic design verification has not been targeted yet. Moreover, in the field of service-oriented manufacturing various process models exist for service orchestration [12, 24, 21]. While some of these models define execution semantics, the models typically are used during technical process implementation rather than specification and verification. Then, according to IEC 61131-3 [2] for developing field level control software Sequential Function Charts provide a graphical programming language supporting the modeling of software processes. Also, state charts have been developed for the specification of control software [26]. Though providing formal semantics and verification capabilities, these approaches are not applied for technical process specification. In a nutshell, a variety of process specification languages have been defined for different purposes. However, the approaches are either not suited for technical process specification or do not provide necessary formal semantics.

Therefore, in Section 2 a dedicated formal approach to technical process specification and verification for automated production systems is presented. In particular, the approach allows developing both the process specification and the implementation independent of each other while achieving their semantic integration for automatic verification at design- and runtime. The applicability of this approach is demonstrated based on a lab sized manufacturing system with exemplified design process in Section 3. The demonstrator is reduced in

⁵ <http://www.esterel-technologies.com/products/scade-suite/>

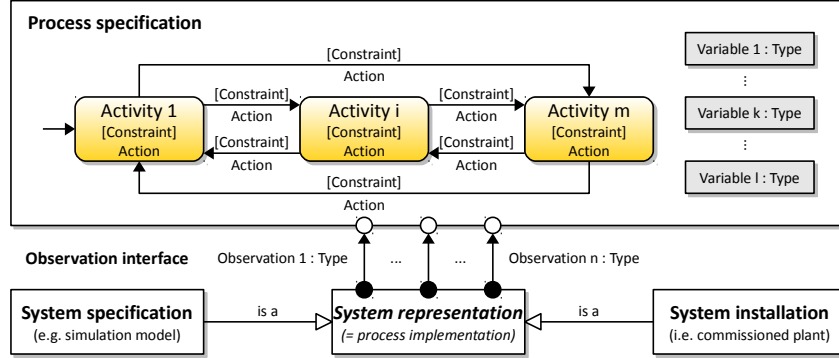


Fig. 1. Overview of the concepts and relations including process specification and system representation connected through an observation interface.

size and complexity. Nevertheless, it is suitable for indicating the applicability for a basic class of manufacturing systems in a first step. It is shown that the verification can lead to an early detection of an exemplary design flaws. The section concludes with a summary of experiences gained during executing the case study. Finally, the paper summarizes the findings and indicates necessary future work.

2 A Formal Approach

To explain our formal approach to technical process specification and verification we first introduce a number of basic concepts and relations (Section 2.1), before going into details of their formalization (Section 2.2) and showing possibilities for automatic verification (Section 2.3).

2.1 Terminology

Figure 1 provides an overview of the core concepts and relations: We distinguish between (1) *process specification* and (2) *system representation*, which are connected and semantically integrated by an (3) *observation interface*.

Process Specification As discussed in the previous section, the purpose of the process specification is to describe the expected behavior of automated production systems in a high-level and cross-discipline manner. To achieve this goal, we propose to describe technical processes in terms of (a) *activities*, (b) *observations*, (c) *variables*, (d) *transitions*, (e) *actions* and (f) *constraints*. The terminology and the graphical notation are inspired by UML activity diagrams [5].

Activities are the main concept for structuring the course of action of an automated production system. Activities define time intervals during which production operations can be performed. Examples are moving and manipulating

solid workpieces or mixing and stirring liquids. During activities observations provide information about the system state such as the position and the shape of a workpiece or the temperature and the chemical mixture of a liquid. Furthermore, variables and actions on variables can be used to track information such as the activity duration or cumulative energy consumption. Then, constraints can be used to limit both observations and variable assignments. Consequently, constraints allow one to document necessary conditions that need to hold during activity execution such as the activity duration, required temperature bands of liquids, and workpiece positions. Finally, activities can be switched by means of transitions describing possible activity sequences. Again, constraints over observations and variables are used to define the conditions that need to hold for switching activities. Examples are intermediate locations of workpieces or intermediate mixtures of liquids. At last, transition actions on variables can be used to track information across activities or to reset the tracking state.

The modeling technique is similar to input/output (i.e. I/O) automata [3]. However, the key differences are that the process specification does not include any outputs and the activities include the concept of constraints. Consequently, our notation reflects the needs of technical process engineers more closely.

Observation Interface The purpose of the observation interface is to link process specification and system representation such that automatic process verification becomes feasible. Conceptually, the observation interface is modeled in terms of observation *ports* and *channels*, which is inspired by the FOCUS engineering method and underlying formalism [9]. The process specification defines the observation input ports, while the system representation provides the respective observation output ports. The port coupling is achieved by means of observation channels.

The interface concept allows one to decouple the process specification and the system representation, while achieving their semantic integration. Consequently, the system representation can be exchanged without the need for revising the process specification, e.g. when comparing automated production systems from different providers implementing the same technical process.

System Representation The system representation finally constitutes the actual implementation of the process specification. Implementations typically include a number of mechanical, electrical and software elements. At this point we do not prescribe any particular representation format. Rather, we distinguish two general options both being suited for automated process verification: (a) *System specifications* and (b) *system installations*. System specifications describe the automated production system e.g. using MechatronicUML [7] or SysML4Mechatronics [18], while system installations represent the commissioned automated production systems themselves. However, note that for automated process verification the system representation needs to define an execution semantics and needs to implement the observation interface.

2.2 Formalization

To enable automatic verification we first define the elements of a process specification in terms of mathematical sets and functions reflecting the concepts introduced in the previous section.

Definition 1 (Process specification). *A process specification P is an twelve-tuple $P = (A, M, N, O, V, T, a', v', f_1, f_2, g_1, g_2)$ with*

- a finite set $A = \{a_1, \dots, a_m\}$ of activities with $m \in \mathbb{N}$,
- a finite set $M = \{M_1, \dots, M_n\}$ of observation domains with $n \in \mathbb{N}$,
- a finite set $N = \{N_1, \dots, N_l\}$ of variable domains with $l \in \mathbb{N}$,
- a finite set $O = M_1 \times \dots \times M_n$ of observations from observation domains,
- a finite set $V = N_1 \times \dots \times N_l$ of variable assignments from variable domains,
- a finite set $T \subseteq A \times A$ of transitions,
- an element $a' \in A$ as initial activity,
- an element $v' \in V$ as initial variable assignments,
- an activity constraint function $f_1 : A \rightarrow \mathcal{P}(O \times V)$,
- an activity action function $f_2 : A \times O \times V \rightarrow V$,
- a transition constraint function $g_1 : T \rightarrow \mathcal{P}(O \times V)$ and
- a transition action function $g_2 : T \times O \times V \rightarrow V$

such that the transition set T does not include self-transitions for any activity from the activity set A :

$$\forall (a_i, a_j) \in T : a_i \neq a_j \text{ with } 1 \leq i, j \leq m \text{ and } i, j \in \mathbb{N}$$

and only one transition in the transition set T is enabled by the transition constraint function g_1 at a time:

$$\forall a_i \in A : \bigcap_{(a_i, a_j) \in T} g_1((a_i, a_j)) = \emptyset \text{ with } 1 \leq i, j \leq m \text{ and } i, j \in \mathbb{N}$$

Note that a process specification P resembles a directed graph with vertices A and edges T . Furthermore, the constraint functions f_1 and g_1 specify possible combinations of observations $o \in O$ and variables $v \in V$, which are accepted while performing activities $a \in A$ or which are required in order to switch activities using transitions $(a_i, a_j) \in T$ with $1 \leq i, j \leq m$ and $i, j \in \mathbb{N}$. Finally, the action functions f_2 and g_2 define how to derive new variable assignments $v' \in V$ from observations $o \in O$ and previous variables assignment $v \in V$ during activities $a \in A$ and transitions $(a_i, a_j) \in T$ with $1 \leq i, j \leq m$ and $i, j \in \mathbb{N}$.

Based on process specification P we define the concept of *observation traces* reflecting the input of the process specification and the output of the system representation respectively.

Definition 2 (Observation traces). *An observation trace τ_n for process specification $P = (A, M, N, O, V, T, a', v', f_1, f_2, g_1, g_2)$ is a finite or infinite sequence:*

$$\tau = (\omega_i)_{i=0}^n$$

with $\omega_i \in O$ representing observations and $n \in \mathbb{N} \cup \{\infty\}$ representing the length of the sequence.

Consequently, observation traces provide a record of system execution. Note that in theory observation traces can be of infinite length, which is important for exhaustive model checking as discussed in the following section.

Based on process specification P and the observation trace τ_n with $n \in \mathbb{N} \cup \{\infty\}$ we define the formal *process execution* semantics determining the order of action execution, constraint evaluation and activity switching.

Definition 3 (Process execution). *A process execution π_n of process specification $P = (A, M, N, O, V, T, a', v', f_1, f_2, g_1, g_2)$ and observation trace $\tau_n = (\omega_k)_{k=0}^n$ with $n \in \mathbb{N} \cup \{\infty\}$ is a finite or infinite sequence:*

$$\pi_n = ((\alpha_k, \omega_k, \phi_k, \beta_k))_{k=0}^n$$

with $\alpha_k \in A$, $\omega_k \in O$, $\phi_k \in V$ and $\beta_k \in \mathbb{B}$ such that the sequence starts with the initial activity a' and the initial variable assignments v' :

$$\alpha_0 = a', \phi_0 = v'$$

and given the domains of sequence indices D_1 and D_2 separating between the finite and the infinite case:

$$\begin{aligned} (n = \infty \Leftrightarrow D_1 = \mathbb{N}) \wedge (n \neq \infty \Leftrightarrow D_1 = \{k \in \mathbb{N} \mid k < n\}) \\ (n = \infty \Leftrightarrow D_2 = \mathbb{N}) \wedge (n \neq \infty \Leftrightarrow D_2 = \{k \in \mathbb{N} \mid k \leq n\}) \end{aligned}$$

the transitions between the activities are included in the transition set T and supported by the transition constraint function g_1 :

$$\forall k \in D_1 : \alpha_k \neq \alpha_{k+1} \Leftrightarrow (\alpha_k, \alpha_{k+1}) \in T \wedge (\omega_k, \phi_k) \in g_1((\alpha_k, \alpha_{k+1}))$$

and the variables are updated according to the activity action function f_2 and transition action functions g_2 :

$$\forall k \in D_1 : \phi_{k+1} = \begin{cases} f_2(\alpha_{k+1}, \omega_{k+1}, \phi_k) & \text{if } \alpha_k = \alpha_{k+1} \\ f_2(\alpha_{k+1}, \omega_{k+1}, g_2((\alpha_k, \alpha_{k+1}), \omega_k, \phi_k)) & \text{if } \alpha_k \neq \alpha_{k+1} \end{cases}$$

and the boolean variables β_k contain the results of the activity constraint function f_1 respectively:

$$\forall k \in D_2 : \beta_k = \text{true} \Leftrightarrow (\omega_k, \phi_k) \in f_1(\alpha_k)$$

Essentially, a process execution π_n with $n \in \mathbb{N} \cup \{\infty\}$ extends the observation trace τ_n with activity, variable assignments and activity constraint information. Note that transitions $(\alpha_k, \alpha_{k+1}) \in T$ with $k \in D_1$ occur between elements of the sequence π_n based on the observations $\omega_k \in O$ and variable assignments $\phi_k \in V$ of the former time point. Also, the effects of the transition actions $\phi'_{k+1} = g_2((\alpha_k, \alpha_{k+1}), \omega_k, \phi_k)$ with $k \in D_1$ on the variables $\phi_{k+1} \in V$ are hidden by the effects of the activity actions $f_2(\alpha_{k+1}, \omega_{k+1}, \phi'_{k+1})$. This design decision has been taken such that for every time point an activity as well as the activity constraints can be determined.

Finally, based on the process specification P , the observation trace τ_n and the process execution π_n the *process satisfaction* condition can be defined, determining whether an observation trace satisfies a process specification or not.

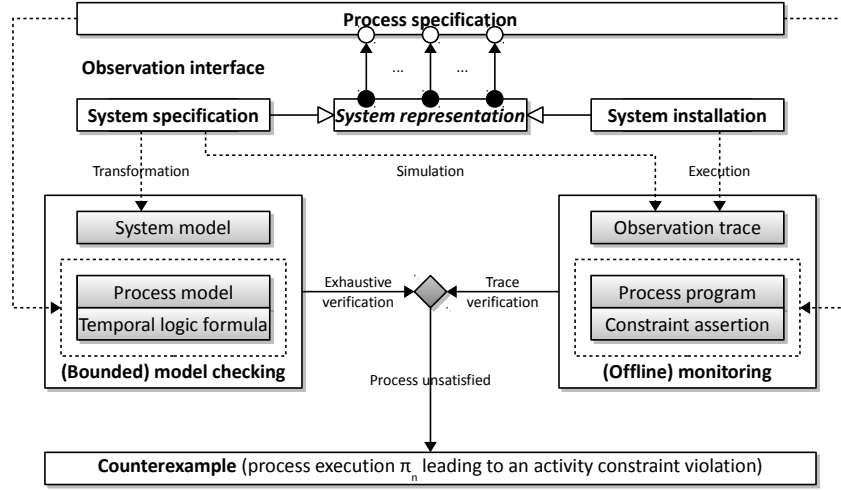


Fig. 2. Verification options including model checking of system specifications, offline monitoring of system specifications and installations and counterexamples.

Definition 4 (Process satisfaction). *Given some process specification $P = (A, M, N, O, V, T, a', v', f_1, f_2, g_1, g_2)$, an observation trace $\tau_n = (\omega_k)_{k=0}^n$ and the respective process execution $\pi_n = (\alpha_k, \omega_k, \phi_k, \beta_k)_{k=0}^n$:*

$$\tau_n \text{ satisfies } P \Leftrightarrow \forall k \in D : \beta_k = \text{true}$$

with $n \in \mathbb{N} \cup \{\infty\}$ defining the sequence length and D representing the finite or infinite set of sequence indices:

$$(n = \infty \Leftrightarrow D = \mathbb{N}) \wedge (n \neq \infty \Rightarrow D = \{k \in \mathbb{N} : k \leq n\})$$

Consequently, the process specification P remains unsatisfied in case activity constraints $(\omega_k, \phi_k) \in f_1(\alpha_k)$ with $k \in D$ are violated, which are used to determine β_k . Note that a system specification satisfies a process specification P in case all possible observation traces τ_n with $n \in \mathbb{N} \cup \{\infty\}$ satisfy P .

2.3 Verification

Different verification techniques exist that allow one to check whether a process specification is fulfilled by incoming observation traces. Figure 2 shows the verification options and how the models and the properties for the verification tools can be derived.

In case an adequate system specification is given, model checking can be used for exhaustive verification of the process specification (i.e. with respect to all possible traces of the system). The formal model for the model checker is built from the system specification, the observation interface and the process

specification. Meanwhile, the formal property to be verified determines that the variable b_k remains always true. This way all finite and infinite observation traces are verified or a counterexample π_n is returned leading to an activity constraint violation in the last step. However, the application of model checking might not be feasible for some systems due to the complexity of the state space. One possibility to overcome this limitation is to apply bounded model checking instead. However, bounded model checking is limited to verification of finite traces with predefined length only, while the verification still is exhaustive (i.e. with respect to all possible traces of the system with predefined length).

Besides model checking other verification options exist which are applicable in case the formal system model cannot be built (i.e. no adequate system specification is available) or exhaustive verification is not feasible (i.e. the state space is too complex even for bounded model checking). In such cases we rely on observation traces only, e.g. obtained by simulation of the system specification or execution of the system installation. Checking whether an observation trace meets the process specification can be performed using online or offline monitoring [22]. As illustrated in the Figure 2, in our case the recorded executions are represented by the observation traces and the monitor is build from the process specification. The monitor verifies the same property as for model checking, i.e. the variable b_k must remain always true. Process monitoring in combination with simulation of the system specification or execution of the system installation requires less expertise, time and memory as compared to model checking, but verifies the process specification for selected finite observation traces only. While this represents a drawback compared to model checking of system specifications, monitors are suited particularly well for verification of system installations.

3 An Academic Case Study

In the following we apply the proposed approach to the pick and place unit (PPU) [25, 17], a bench-scale lab demonstrator of a manufacturing system (cp. Figure 3). The system consists of a *stack* for storing cylindrical workpieces (WPs), a *stamp* for stamping WPs, a *sorter* including a *conveyor*, two pneumatic *cylinders* and three *ramps* for transporting, sorting and storing WPs, as well as a *crane* for transporting WPs between the previous stations.

For the study the following requirements have been specified: The PPU must handle two types of WPs, *plastic* and *metallic*. During the process the latter should be *stamped*, while the former should be left *unstamped* (which is a common scenario for example in waste management). Moreover, both kinds of WPs should be transported from *stack* to *ramp 3* location within at most 25 seconds. Note that *ramp 1* and *ramp 2* location remain unused because in this study the PPU is not required to sort the WPs.

In the following we describe the developed process specification before elaborating on the system specification. Further, we show how in the given case model checking helps to uncover a design flaw in the system specification and prove its subsequent correction.

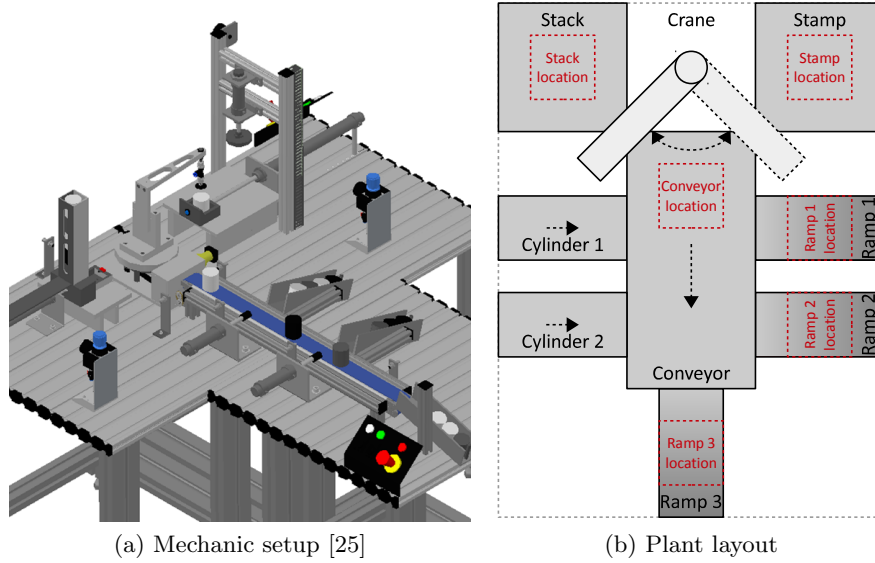


Fig. 3. The pick and place unit (PPU) bench-scale lab demonstrator.

3.1 Process Specification

The process specification for the PPU is depicted in Fig. 4. During design, it was decided to decompose the entire process into six activities and four observations. The activities are *wait*, *pivot plastic WPs*, *pivot metal WPs*, *stamp metal WPs*, *pivot metal WPs* again, and *transport* both types of WPs. In contrast, the observations describe WPs at *stack*, *stamp*, *conveyor*, and *ramp 3* location.

As shown in the process specification, the system is required to start with the *wait* activity. Then two transitions are defined, one for unstamped plastic and the other for unstamped metallic WPs at the stack location. When observing a plastic WP, the PPU switches to the *pivot plastic* activity for a maximum duration of 5 seconds. The activity ends as soon as an unstamped plastic WP is observed at the conveyor location. In contrast, for metallic WPs, a separate *pivot metal* activity is executed for a maximum duration of 10 seconds, and ends as soon as unstamped metallic WPs are observed at the stamp location. Then, the *stamp metal* activity is executed for a maximum duration of 5 seconds, which ends when stamped metallic WPs are observed at the stamp location. Subsequently, again a *pivot metal* activity is triggered for a maximum duration of 5 seconds, which ends exactly when stamped metallic WPs are observed at the conveyor location. Then, independent of the WP type and state the *transport* activity is started for a maximum duration of 5 seconds, until a WP is observed at the ramp 3 location.

Overall the process ensures that both plastic and metallic WPs are processed in at least 25 seconds of time. In case a plastic WP is observed, the sequence of activities lasts at most 15 seconds. In case a metallic WP is observed, the

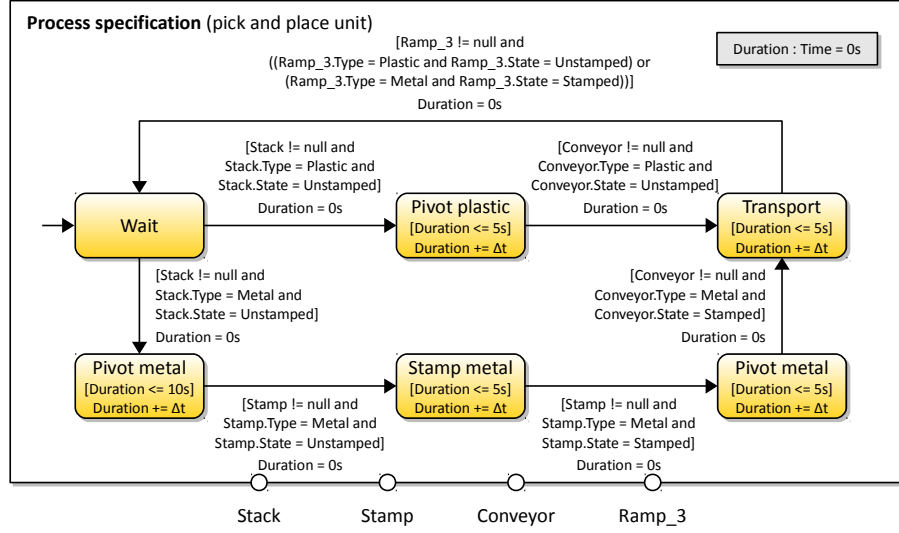


Fig. 4. Process specification for the PPU including different handling of plastic and metallic WPs.

sequence lasts at most 25 seconds instead. Also, in accordance to the requirements at ramp 3 location only unstamped plastic or stamped metallic WPs can be observed.

3.2 System Specification

For this study we decided to use AutoFOCUS⁶ to model the system specification. AutoFOCUS is a reference implementation of the FOCUS formalism [9] and comes with NuSMV⁷ integration [11] for model checking. FOCUS describes systems in terms of components, input/output ports, channels and input/output automata. The advantage of the tool chain is that the system specification can be verified with respect to the process specification. On the downside, the modeling technique is tailored to pure message exchange between components, which also has to be used to describe physical phenomena. However, it should be noted that extensions of FOCUS exist adding dense time [8] and continuous time [10] and spatial [16] phenomena. Though being more suitable for describing physical phenomena, these extensions are not used in the study as they are not supported by the reference implementation yet.

The system specification for the pick and place unit is shown in Figure 5. Due to system complexity we only focus on a part of the system specification including the *workpiece* (i.e. plastic or metallic WP), the *crane* to perform the

⁶ <http://af3.fortiss.org/>

⁷ <http://nusmv.fbk.eu/>

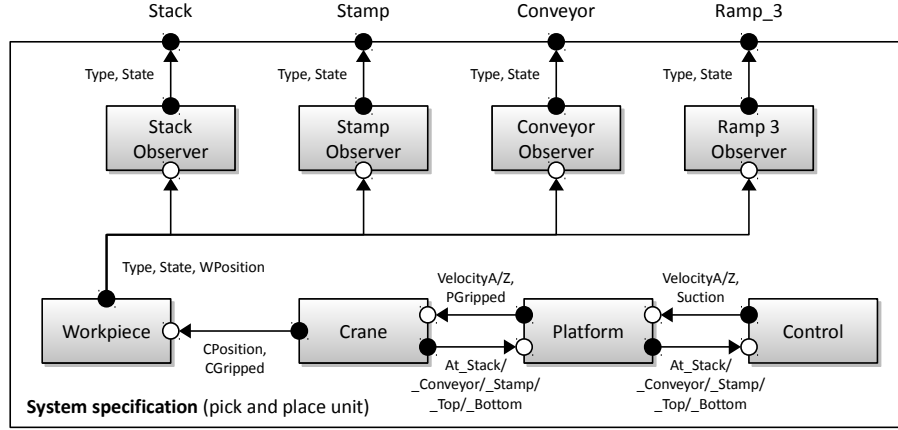


Fig. 5. Excerpt of a simplified system specification for the PPU including novel observer components.

pivot activities (see Section 3.1), a *platform*, a *control* and several *observer* components. Note that the system specification assumes that only one WP is being processed by the PPU at a time, which is in accordance with the process specification.

Workpiece The workpiece component is responsible for modeling the workpiece type (plastic or metallic), state (unstamped or stamped) and position (angular and lift). The component defines two input ports (*crane position* and *suction*) as well as three output ports (*type*, *state* and *position*). The inputs are provided by the crane component, while the outputs are delivered to the observer components. Figure 6a shows the automaton specification of the input/output behavior of the workpiece component. The behavior is described in terms of two states: (1) *Not gripped* and (2) *gripped*. In case the workpiece is gripped by the crane, the workpiece position (i.e. **WPosition**) is locked to the crane position (i.e. **CPosition**). Otherwise the workpiece position remains constant. The workpiece goes into the gripped state if the workpiece position is equal to the crane position and the gripped input is turned on. On the other hand, the workpiece goes back into the not gripped state as soon as the gripped input is turned off.

Note that the automaton does not include the type and state outputs of the workpiece, which are irrelevant for the interaction between workpiece and crane. Obviously, the type (i.e. plastic or metallic) remains constant, while the state depends on the interaction with the stamp, which is not focused in this presentation.

Crane The crane component is responsible for modeling the crane position (angular and lift) as well as the suction and position sensor outputs. The component

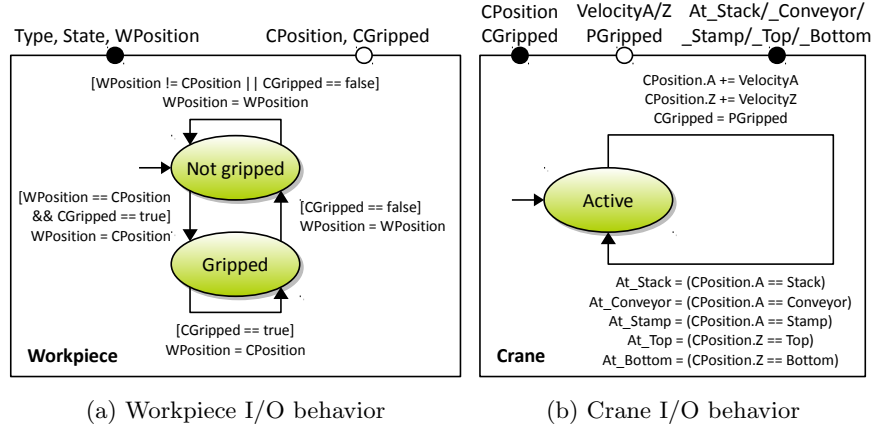


Fig. 6. Automaton specification of the input/output behavior for the workpiece and crane components.

defines three input ports (*angular velocity*, *lift velocity* and *suction*) as well as seven output ports (*crane position*, *suction*, *at stack*, *at conveyor*, *at stamp*, *at top* and *at bottom*). The inputs are provided by the platform component, while the first two outputs are forwarded to the workpiece component and the last five outputs are delivered back to the platform component. Figure 6b shows the automaton specification of the I/O behavior of the crane component. The behavior is described by means of a single state: *Active*. In this state angular and lift velocity are added to the crane position, while the suction input is forwarded to the workpiece unchanged and the position sensors fire selectively.

Platform The platform component is responsible for modeling the physical connection between crane and control component. Consequently, the velocity and suction values are forwarded to the crane, while the position sensor values are delivered back to the control. An automaton specification of the I/O behavior is omitted here, however it should be noted that the platform introduces a message delay between crane and control. Technically, the delay is caused by analog-digital converters, communication buses and programmable logic controller execution semantics.

Control The control component is responsible for adjusting crane velocity and suction based on the position sensor inputs. Again, due to space limitation the automaton specification of the I/O behavior is omitted. The automaton switches between different *pivot*, *lift* and *wait* states controlling angular and lift velocities as well as suction respectively. State switches occur based on the position sensor inputs from the crane as well as further information about the workpiece state (omitted in the presented system specification).

		Step	1	2	...	30	31	...	102	103
System model	Workpiece	WPosition	{A: 0, Z: 0}	{A: 0, Z: 0}	...	{A: 65, Z: 1}	{A: 70, Z: 1}	...	{A: 210, Z: 0}	{A: 210, Z: 0}
		...	...	...	...	...	...	...	...	...
	Crane	CPosition	{A: 0, Z: 0}	{A: 0, Z: 0}	...	{A: 65, Z: 1}	{A: 70, Z: 1}	...	{A: 210, Z: 0}	{A: 210, Z: 0}
		OSuction	false	false	...	true	true	...	false	false
	...	...	...	...	...	...	...	...	...	...
	...	...	...	...	...	...	...	...	...	...
Process model	Activity		Wait	Pivot metal	...	Pivot metal	Pivot metal	...	Pivot metal	Pivot metal
	Observations	Stack	{Type: Metal}	null	...	null	null	...	null	null
		...	...	...	...	...	...	...	...	...
	Variables	Duration	0.0s	0.1s	...	3.0s	3.1s	...	10.0s	10.1s
Activity Constraint			true	true	...	true	true	...	true	false

Fig. 7. Counterexample returned by the model checker leading to an activity constraint violation in the last step.

Observer Finally, the observer components are responsible for defining the observation streams necessary to connect the system specification to the process specification (see Section 3.1). Based on the workpiece position the observers decide whether to forward the input message including workpiece type and state to the respective observation output port.

3.3 Model Checking

Due to the selected tool chain (i.e. AutoFOCUS and NuSMV) we are able to perform exhaustive verification of the system specification with respect to the process specification using for example bounded model checking. Therefore, as described in [11] and Section 2.3 both the system specification and the process specification are translated into NuSMV modules.

For verification the workpiece type is set to metallic, the workpiece state is set to unstamped and the workpiece and crane positions are set to the stack location. Then, for bounded model checking the analysis depth is set to 110 steps (covering the execution of the first pivot metal activity). Further, to indicate verification performance an analysis timeout of 50 seconds is used. Given these settings NuSMV fails to verify the absence of activity constraint violations within the timeout returning the counterexample shown in Figure 7. In the counterexample the process model correctly switches to the pivot activity after detecting unstamped metallic WPs at the stack location. Also, the WP transport seems to work correctly after suction is turned on. However, the WP is moved past the stamp location before the crane places the WP down and turns suction off. Consequently, the activity post condition (i.e. unstamped metallic WP at the stamp location) does not become true and the duration constraint is violated. After further examination of the counterexample the cause of the problem is identified: As illustrated in Figure 8a the delay introduced by the

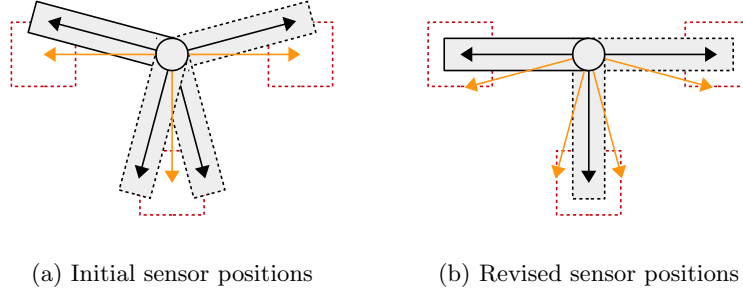


Fig. 8. Geometric explanation of the design flaw and its correction through manipulation of sensor positions.

platform component leads to a deviation between crane position sensor angles and final crane angles, which is why the WP moves past the stamp location.

To correct the design flaw, the angular position sensors are slightly displaced as shown in Figure 8b. Also, a sensor is added to the conveyor location as the crane approaches the location from two sides. Finally, the control component is adjusted to accommodate for the novel input. Subsequently, given the same settings NuSMV is able to verify the absence of activity constraint violations. However, note that only the first pivot metal activity is considered currently, while an exhaustive verification of the entire process has not been achieved yet.

4 Conclusion & Outlook

In this paper we have introduced a dedicated formal approach to technical process specification and verification for automated production systems. In Section 2 we developed the underlying terminology (in compliance with existing approaches [27, 5]) and provided the necessary mathematical definitions before describing verification options both at design- and at runtime. Then, in Section 3 we showed how to apply the proposed approach to a bench-scale manufacturing system. The case study included the formal process specification as well as an excerpt of the system specification in AutoFOCUS. Moreover, model checking with NuSMV was used to uncover a design flaw caused by signal delays. Finally, one possibility to resolve the design flaw was discussed.

The case study showed the effectiveness of using an observation interface to decouple and semantically integrate process specification and system representation. In particular, high-level observations can be defined early during process specification, while a translation to these observations can be achieved easily later within the system’s design to enable automatic verification. Furthermore, we have exploited the concept of variables and actions to model and constrain the duration of activities. Similarly, other performance characteristics such as energy consumption can be constrained. Finally, both model checking and simulation monitoring provide powerful tools for design-time verification, while monitoring

also provides the link to system commissioning and operation. Overall, through the small set of concepts and different verification options we are able to provide a flexible basis for technical process specification and verification.

However, on the downside both actions and constraints currently require lengthy textual notation, which process engineers are not accustomed to. To this end we are working on graphical notations as well as the inclusion of language-level patterns. Moreover, we are working on extensions for process composition and synchronization to support parallel process execution. Also, we are investigating continuous-time rather than discrete-time process specifications. Then, we are exploring possibilities for process quality monitoring as compared to constraint monitoring. Finally, we are studying exhaustive verification performance as well as scenario-based testing as alternative, scalable verification strategy.

References

1. VDI/VDE 3682: Formalised process description (09 2005)
2. Programmable logic controllers – part 3: Programming languages (2009), IEC Standard 61131-3
3. Attie, P., Lynch, N.: Dynamic input/output automata: A formal model for dynamic systems. In: Larsen, K., Nielsen, M. (eds.) *CONCUR 2001 Concurrency Theory*, Lecture Notes in Computer Science, vol. 2154, pp. 137–151. Springer Berlin Heidelberg (2001)
4. Balser, M., Bäuml, S., Knapp, A., Reif, W., Thums, A.: Interactive Verification of UML State Machines. In: Davies, J., Schulte, W., Barnett, M. (eds.) *6th International Conference on Formal Engineering Methods (ICFEM'04, Proceedings)*. Lecture Notes in Computer Science, vol. 3308, pp. 434–448. Springer (2004)
5. Bastos, R., Ruiz, D.: Extending uml activity diagram for workflow modeling in production systems. In: *System Sciences, 2002. HICSS. Proceedings of the 35th Annual Hawaii International Conference on*. pp. 3786–3795 (Jan 2002)
6. Bauer, A., Leucker, M., Schallhart, C.: Runtime verification for ltl and tltl. *ACM Trans. Softw. Eng. Methodol.* 20(4), 14:1–14:64 (Sep 2011)
7. Becker, S., Brenner, C., Dziwok, S., Gewering, T., Heinzemann, C., Pohlmann, U., Priesterjahn, C., Schäfer, W., Suck, J., Sudmann, O., Tichy, M.: The mechatronicuml method - process, syntax, and semantics. Tech. Rep. tr-ri-12-318, Software Engineering Group, Heinz Nixdorf Institute University of Paderborn (2012)
8. Broy, M.: System behaviour models with discrete and dense time. In: Chakraborty, S., Eberspacher, J. (eds.) *Advances in Real-Time Systems*. pp. 3–25. Springer (2012)
9. Broy, M., Stølen, K.: Specification and development of interactive systems: Focus on streams, interfaces and refinement. Springer (2001)
10. Campetelli, A.: Dynamic Sampling for FOCUS Hybrid Components. In: Ölveczky, P.C., Artho, C. (eds.) *3rd International Conference on Circuits, System and Simulation (ICCSC 2013)*. vol. 3(5), pp. 402–406. International Journal of Modeling and Optimization (2013)
11. Campetelli, A., Hölzl, F., Neubeck, P.: User-friendly Model Checking Integration in Model-based Development. In: *24th International Conference on Computer Applications in Industry and Engineering* (2011)
12. Cândido, G., Barata, J., Colombo, A.W., Jammes, F.: Soa in reconfigurable supply chains: A research roadmap. *Engineering applications of artificial intelligence* 22(6), 939–949 (2009)

13. Christiansen, L., Fay, A., Opgenoorth, B., Neidig, J.: Improved diagnosis by combining structural and process knowledge. In: Emerging Technologies Factory Automation (ETFA), 2011 IEEE 16th Conference on. pp. 1–8 (Sept 2011)
14. Damm, W., Harel, D.: LSCs: Breathing Life into Message Sequence Charts 19(1), 45–80 (2001)
15. Dijkman, R.M., Dumas, M., Ouyang, C.: Semantics and analysis of business process models in bpmn. *Inf. Softw. Technol.* 50(12), 1281–1294 (Nov 2008)
16. Hummel, B.: Integrated Behavior Modeling of Space-Intensive Mechatronic Systems. Dissertation, Technische Universität München, München (2011)
17. Institute of Automation and Information Systems, Technische Universität München: The pick and place unit demonstrator for evolution in industrial plant automation. Online available: <http://www.ppu-demonstrator.org> (2014)
18. Kernschmidt, K., Vogel-Heuser, B.: An interdisciplinary sysml based modeling approach for analyzing change influences in production plants to support the engineering. In: IEEE International Conference on Automation Science and Engineering (CASE). Madison, USA (2013)
19. Kohn, A., Reif, J., Wolfenstetter, T., Kernschmidt, K., Goswami, S., Krcmar, H., Brodbeck, F., Vogel-Heuser, B., Lindemann, U.: Improving common model understanding within collaborative engineering design research projects. In: 4th International Conference on Research into Design (2013)
20. Li, F., Bayrak, G., Kernschmidt, K., Vogel-Heuser, B.: Specification of the requirements to support information technology-cycles in the machine and plant manufacturing industry. In: 14th IFAC Symposium on Information Control Problems in Manufacturing (2012)
21. Loskyll, M., Schlick, J., Hodek, S., Ollinger, L., Gerber, T., Pirvu, B.: Semantic service discovery and orchestration for manufacturing processes. In: Emerging Technologies & Factory Automation (ETFA), 2011 IEEE 16th Conference on. pp. 1–8. IEEE (2011)
22. Maler, O., Nickovic, D.: Monitoring temporal properties of continuous signals. In: Lakhnech, Y., Yovine, S. (eds.) *Formal Techniques, Modelling and Analysis of Timed and Fault-Tolerant Systems*, Lecture Notes in Computer Science, vol. 3253, pp. 152–166. Springer Berlin Heidelberg (2004)
23. Schinz, I., Toben, T., Mrugalla, C., Westphal, B.: The Rhapsody UML Verification Environment. In: 2nd International Conference on Software Engineering and Formal Methods. pp. 174–183. IEEE Computer Society (2004)
24. Shen, W., Hao, Q., Wang, S., Li, Y., Ghenniwa, H.: An agent-based service-oriented integration architecture for collaborative intelligent manufacturing. *Robotics and Computer-Integrated Manufacturing* 23(3), 315–325 (2007)
25. Vogel-Heuser, B., Legat, C., Folmer, J., Feldmann, S.: Researching evolution in industrial plant automation: Scenarios and documentation of the pick and place unit. Technical Report TUM-AIS-TR-01-14-02, Institute of Automation and Information Systems, Technische Universität München (2014), <https://mediatum.ub.tum.de/node?id=1208973>
26. Witsch, D., Vogel-Heuser, B.: PLC-statecharts: An approach to integrate UML-statecharts in open-loop control engineering – aspects on behavioral semantics and model-checking. In: 18th IFAC World Congress (2011)
27. Zor, S., Leymann, F., Schumm, D.: A Proposal of BPMN Extensions for the Manufacturing Domain. In: Proceedings of the 44th CIRP Conference on Manufacturing Systems (ICMS 2011); Madison, Wisconsin, June 1-3, 2011. pp. 1–6. - (January 2011)